02207 PyUVM Compendium

Version 1.1.3.0

SyoSil ApS

Oct 18, 2026

CONTENTS:

1 Instance Hierarchy **1**
- 1.1 Components, parents, and children . . . 1
- 1.2 Instance paths . . . 2
- 1.3 Wildcard path patterns . . . 3

2 ConfigDB **4**
- 2.1 Context and instance paths . . . 4
- 2.2 ConfigDB set() . . . 5
- 2.3 ConfigDB get() . . . 5
- 2.4 Why use the ConfigDB instead of direct assignments? . . . 5

3 Factory **7**
- 3.1 Type registration . . . 7
- 3.2 Creating objects and components . . . 7
- 3.3 Overrides . . . 8
  - 3.3.1 Type overrides . . . 8
  - 3.3.2 Instance overrides . . . 9

4 TLM Analysis Ports and Exports **10**
- 4.1 Analysis Port and Export in PyUVM . . . 10
- 4.2 Classes with Build-In Analysis Ports/Exports . . . 11
  - 4.2.1 UVM Subscriber . . . 11
  - 4.2.2 TLM analysis FIFO . . . 11

CHAPTER

ONE

INSTANCE HIERARCHY

The `uvm_components` of a pyuvm testbench form an instance hierarchy. It describes the parent-child relationships between all `uvm_components` in a testbench.

Objects derived from `uvm_object` are not part of the instance hierarchy.

1.1 Components, parents, and children

When a `uvm_component` is created using the factory with the `create()` class method, the `name` and `parent` of the component are set.

```
class default_test(uvm_test):
   ...
   def build_phase(self):
      ...
      self.env = env.create(
          name   = "env",
          parent = self,
      )
```

This creates a environment component with the name `env` as a child of the test. The instance of the test in a pyuvm testbench is created by `uvm_root` and is always named `uvm_test_top`.

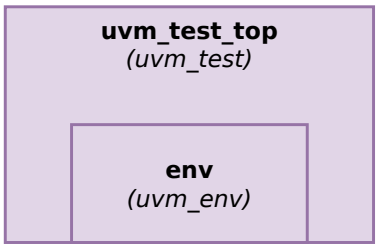


Fig. 1.1: The `env` component is a child of `uvm_test_top`.

1.2 Instance paths

An instance path is the dot-separated sequence of instance names from the top component (`uvm_test_top`) to a particular component.

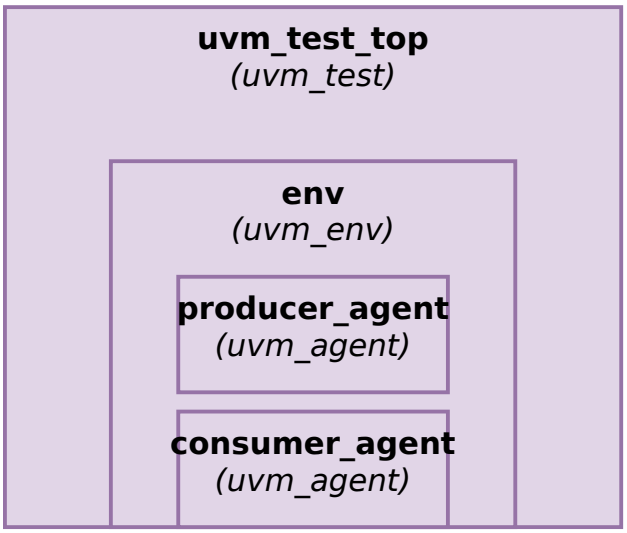


Fig. 1.2: Deeper nested instance hierarchy with two `uvm_agents`.

The example from above creates this hierarchy:

```
uvm_root
└── uvm_test_top
    └──  env
       ├── producer_agent
       └── consumer_agent
```

The corresponding full instance paths are:

Component	Full instance path
test	`uvm_test_top`
environment	`uvm_test_top.env`
producer agent	`uvm_test_top.env.producer_agent`
consumer agent	`uvm_test_top.env.consumer_agent`

To get the name or full instance path of a component the following methods are available.

- `get_name()` returns only the local instance name.
- `get_full_name()` returns the full instance path.

1.3 Wildcard path patterns

Path-based methods such as `ConfigDB`'s `set()` and `uvm_factory`'s `override_instance_by_<type/name>` can match a concrete instance path against a wildcard pattern[1]. This addresses a region of the hierarchy without requiring a reference to every component in that region. Most importantly this allows the use of `*` which matches against everything. This allows a instance override for example to target all components within the environment:

```
# In the env: replace all SequenceItem classes with NewSequenceItem
uvm_factory().set_inst_override_by_type(
    original_type  = SequenceItem,
    override_type  = NewSequenceItem,
    full_inst_path = "uvm_test_top.env.*",
)
```

Similarly the test can store a value in the `ConfigDB` for all descendants of `env`:

```
ConfigDB().set(
   context    = self,
   inst_name  = "env.*",
   field_name = "timeout_cycles",
   value      = 100,
)
```

[1] Pyuvm specifically uses fnmatch for matching wildcard patterns.

CHAPTER

TWO

CONFIGDB

The configuration database, `ConfigDB`, is a shared, path-based database. Components can store a value using `set()`, and retrieve it using `get()`. The value is identified by a field name, similar to a dictionary, and an instance path. A key value pair is only 'visible' to a single (or multiple) components.

In pyuvm the `ConfigDB` is a singleton object. `ConfigDB()` always accesses the same database object.

2.1 Context and instance paths

Both `set()` and `get()` take two arguments to resolve the effective lookup path.

1. `context`, an object of type `uvm_component`
2. `inst_name`, a string

The effective lookup path is resolved from the full instance path of `context` with the `inst_name` appended. So for a `ConfigDB` call with

- `context`: `sat_default_test` (full instance path: `"uvm_test_top"`)
- `inst_name`: `"env"`

Produces `"uvm_test_top.env"` as the effective lookup path.

The `context` should almost always be the current component referenced by `self`. Then, from the perspective of the `context`, `inst_name` selects the child or descendant to configure. From the receiving component's perspective (again passing itself as `context`) using `inst_name=""` selects its own full instance path. For example, if `env` is a child of `uvm_test_top`:

```
# In the test (uvm_test_top):
ConfigDB().set(
    context    = self,
    inst_name  = "env",
    field_name = "cfg",
    value      = self.cfg,
)

# In the environment (uvm_test_top.env):
self.cfg = ConfigDB().get(
    context    = self,
    inst_name  =  "",
    field_name ="cfg",
)
```

Both calls resolve to `uvm_test_top.env`, so the environment retrieves the config. Using itself as context means that the environment does not need to specify its full path or know the name of the test above it.

`set()` also accepts wildcards in the `inst_name` see *Wildcard path patterns*.

2.2 ConfigDB set()

```
def set(context, inst_name, field_name, value)
```

Stores a value under `field_name` at the path or path pattern formed from `context` and `inst_name`. The `inst_name` is relative to the context component's full instance path. With `context=None`, it is a full path or path pattern from the root.

Arguments:

- `context`: A `uvm_component` used as the base for the path, or `None` to use the root context.
- `inst_name`: An instance path relative to `context`. Use `""` for the context component itself. Wildcards are allowed.
- `field_name`: The key at which to store the value.
- `value`: The value to store.
- Returns: `None`.

2.3 ConfigDB get()

```
def get(context, inst_name, field_name, default=default_get)
```

Retrieves a value by matching the concrete path formed from `context` and `inst_name` against the stored paths and path patterns, then looking up `field_name`. The `inst_name` is relative to the context component's full instance path. With `context=None`, it is a full path from the root.

Arguments:

- `context`: A `uvm_component` used as the base for the path, or `None` to use the root context.
- `inst_name`: A concrete instance path relative to `context`, without wildcards.
- `field_name`: The key for the value to retrieve.
- `default`: The value to return if no matching configuration value is found.
- Returns: The stored value, or the supplied `default` if no matching configuration value is found.

2.4 Why use the ConfigDB instead of direct assignments?

A direct assignment such as `self.env.driver.timeout_cycles = 100` requires the test to have access to the driver object and to know which attribute to change. The driver must also exist before the assignment can be made. This couples the test to the Python object structure of the environment.

With the `ConfigDB`, the test specifies a value and the instance path where it should be available. The receiving component retrieves the value under an agreed field name and assigns its own attributes. The value can be stored before the receiving component exists, because the database stores paths rather than looking up component objects when `set()`

is called. This is useful during `build_phase()`, when parents configure components that will be created further down the hierarchy.

The receiving component does not need to know the name of the test or how to navigate the Python references above it. It can retrieve configuration relative to its own instance path, making it easier to reuse in another environment. The test and component must still agree on the field name, the meaning of the value, and a matching instance path.

CHAPTER

THREE

FACTORY

The `uvm_factory` is used to create `uvm_object` and `uvm_component` instances. It implements the factory pattern. Objects are not instantiated directly though their constructor, instead the `create()` method is called, which requests an object of the caller class (type) from the factory. This allows for run time class overrides, i.e. altering the returned type for a specific request.

3.1 Type registration

Classes extending `uvm_object` or `uvm_component` are automatically registered with the factory. *PyUVM* does not require registration macros or a manual registration call. Both `uvm_object` and `uvm_component` implement a `create()` method to request an object of the class type.

3.2 Creating objects and components

The `create()` method returns an object of the class type. It also works for any subclasses of `uvm_object` or `uvm_component`.

The `create()` method for `umv_object` takes only a name:

```
class sat_tb_config(uvm_object)
   ...

self.cfg = sat_tb_config.create(
   name="cfg",
)
```

The `create()` method for `uvm_component` takes a name and parent instance:

```
class sat_tb_env(uvm_component)
   ...

self.tb_env = sat_tb_env.create(
    name   = "env",
    parent = self,
)
```

Calling the class constructor directly bypasses the factory, so factory overrides do not affect that instance.

```
class sat_tb_env(uvm_component)
   ...

# Env not affected by overrides. Do not do this!
self.tb_env = sat_tb_env(
    name   = "env",
    parent = self,
)
```

3.3 Overrides

An override replaces a requested class with another class. The override has to be issued before the affected `create()` call. It does not replace instances that have already been created.

There are two kinds of overrides:

- A type override applies to every factory request for the original class.
- An instance override only applies when the request matches an instance path.

Overrides are resolved as follows:

- Instance overrides take precedence over type overrides.
- Instance overrides are resolved in the order they are registers, meaning the first matching instance override wins.
- Only one type override can exist per type. If a new type override is registered it takes precedent.

3.3.1 Type overrides

```
def set_type_override_by_type(original_type, override_type, replace=True)
```

Arguments:

- `original_type`: The class to replace.
- `override_type`: The class to create instead.
- `replace`: Whether an existing type override may be replaced.
- Returns: `None`.

For example, this replaces every factory request for `sat_tb_default_seq` with `sat_tb_random_seq`:

```
uvm_factory().set_type_override_by_type(
    original_type = sat_tb_default_seq,
    override_type = sat_tb_random_seq,
)

sequence = sat_tb_default_seq.create(name="top_sequence")
```

Although the request names `sat_tb_default_seq`, `sequence` is an instance of `sat_tb_random_seq`.

3.3.2 Instance overrides

```
def set_inst_override_by_type(original_type, override_type, full_inst_path)
```

Arguments:

- `original_type`: The class to replace.
- `override_type`: The class to create instead.
- `full_inst_path`: The instance path or wildcard path pattern to match.
- Returns: `None`.

The path can contain *wildcard patterns*. For example, this replaces the monitor in every agent below `env`:

```
uvm_factory().set_inst_override_by_type(
    original_type  = default_monitor,
    override_type  = checking_monitor,
    full_inst_path = "uvm_test_top.env.*_agent.monitor",
)
```

The * matches both `producer_agent` and `consumer_agent`, while monitors at other paths are unchanged.

CHAPTER

FOUR

TLM ANALYSIS PORTS AND EXPORTS

Transaction-level modeling (TLM) ports and exports are used to send transaction objects between different parts of the testbench. Analysis ports are used to broadcast transactions between monitors, the reference model and scoreboard.

An analysis connection has two ends:

- An analysis port initiates the `write()` call.
- An analysis export receives the `write()` call and provides its implementation.

But importantly, each analysis port can connect to an arbitrary number of exports. So the analysis port to export connection is a one to many, as can be seen in Fig. 4.1. The connection between a port and export is made in `connect_phase()`.

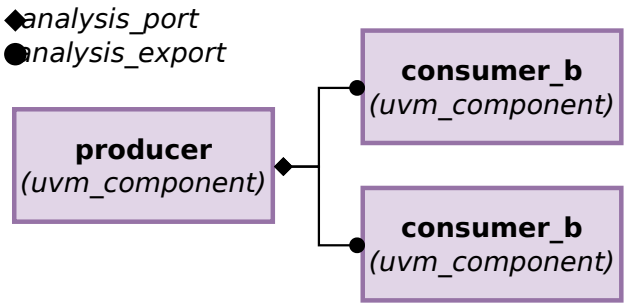


Fig. 4.1: An analysis port connection. One analysis port can connect to many analysis exports.

4.1 Analysis Port and Export in PyUVM

An analysis port is created with:

```
self.analysis_port = uvm_analysis_port(
    name = f"{self.get_name()}_analysis_port",
    parent = self,
)
```

There is typically no need to utilize the factory for analysis ports or exports, as they are never the target of a override.

An analysis port has the method: `write(item)`, which calls the `write(item)` methods of all connected exports. Calling `write()` when the port has no connections is valid, but no component receives the transaction. `write()` only passes a reference to the transaction items. If the originally item is altered all other components will see this changes. Therefore its best to clone the item when received using the `clone()`.

To connect a port to an export in the `connect_phase()`, the `connect(export)` method is used. The `export` passed to `connect()` needs to implement `write()`.

```
def connect_phase(self):
   self.producer.analysis_port.connect(self.ref_model.analysis_export)
```

4.2 Classes with Build-In Analysis Ports/Exports

Some UVM classes have build in ananlysis ports/exports, two are described here:

- `uvm_subscriber`
- `uvm_tlm_analysis_fifo`

4.2.1 UVM Subscriber

`uvm_subscriber` is the base class for components that receive transactions from an `analysis_port`. It creates an `analysis_export` automatically when the subscriber is constructed. The subscriber's analysis export always calls the subclass's `write()` method.

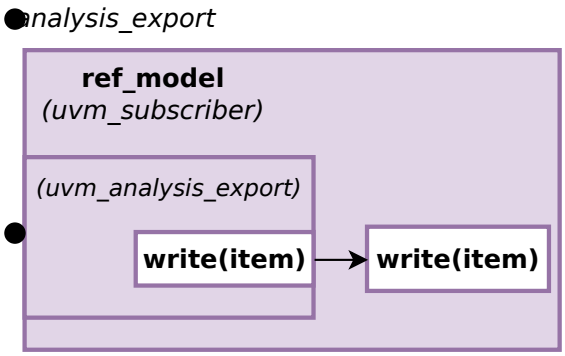


Fig. 4.2: `uvm_subscriber` provides `analysis_export`, which calls the subclass's `write()` method.

Every `uvm_subscriber` subclass must override this method:

```
class ref_model(uvm_subscriber):
    def write(self, item):
        # Process item here.
        pass
```

4.2.2 TLM analysis FIFO

`uvm_tlm_analysis_fifo` is a FIFO with an analysis export. It receives a transaction through `analysis_export.write(item)` and stores the transaction in its queue.

```
self.fifo = uvm_tlm_analysis_fifo(
    name   = "fifo",
    parent = self,
)
```

The FIFO exposes `analysis_export` for an analysis-port connection. The transactions are retrived from the FIFO using the `get()` mehtod. `get()` waits until an item is available and removes that item from the FIFO.

```
item = await self.fifo.get()
```

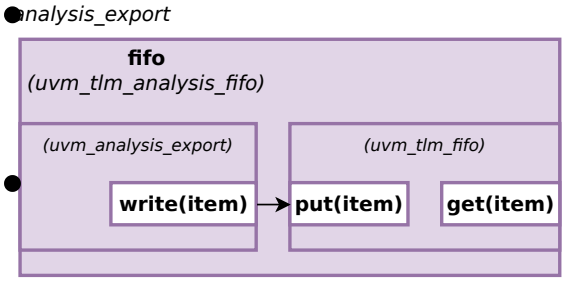


Fig. 4.3: `analysis_export` appends transactions to the analysis FIFO queue.